Promotion Review Tracker: Adding a Field to a Live Form

A worked walkthrough – adding Review Assigned Date to Journal Reviews

2026-11-08

This is a concrete, step-by-step walkthrough of the most common maintenance task on a live Promotion Review Tracker install: adding one new field to a form that already has data in it. It uses a real example – adding a `review_assigned_date` field to the `journal_reviews` form – with the exact commands that were run against a live deployment while writing this guide.

This is a worked instance of the general procedure in `vignette('promotion-tracker-technical-guide')`, Section 10 ("Updating the Data Dictionary After Go-Live"). Read that section first if you have not already; this document does not repeat its warnings, only applies them.

A word on "rebuilding" the database. If your instinct is to re-run `setup_zzedc_from_gsheets()` to pick up a Data Dictionary change, stop. That function calls `initialize_encrypted_database()` internally, and with `overwrite = TRUE` it deletes the existing database file and creates an empty one with a brand-new encryption key and salt – every record anyone has entered is gone, with no recovery path. There is no supported "rebuild in place" operation. What this guide does instead is add the one new column to the running database directly, which takes under a second and touches nothing else.

For a non-technical trial manager: what you can do yourself

If you are a trial manager without database access, you can still do the recordkeeping half of this change yourself in the Google Sheet – you just cannot finish it alone.

What you can do in Google Sheets. Open the Promotion Review Tracker Sheet, go to the `Data_Dictionary` tab, find the rows for `journal_reviews`, and add one new row:

Column	Value
form_code	journal_reviews
field_name	review_assigned_date
field_label	Review Assigned Date
field_type	date
required	FALSE
description	Date the editor assigned this manuscript for review
field_order	35

Use `field_order = 35` so the new field lands between Manuscript ID (30) and Review Date (40) – the gaps of 10 between existing rows exist exactly so a new row can be slotted in without renumbering anything else.

Where it stops being a Sheet-only job. Editing the Sheet does *not*, by itself, make the field appear in the live app. There is no automatic sync from the Sheet to the running form – the Sheet is the durable record of what the form *should* contain, but someone with database access still has to apply the matching change to the live database (Sections 2-3 below). That step takes an admin under a minute. In practice: update the Sheet for recordkeeping and audit purposes, then hand off to whoever manages the deployment to apply it live.

1. Update the Google Sheet first

The Google Sheet's `Data_Dictionary` tab is the durable source of truth: it is what a fresh install or a disaster-recovery rebuild reads from, so an addition that only exists in the live database and not in the Sheet will be silently lost the next time someone provisions a new copy of this study from scratch. Always update the Sheet, even though (as of this writing) there is no supported re-import path that would pick the change up automatically on a live database – Section 2 below is still a separate step.

Open the Sheet (Section 6.1 of the technical guide has the URL and tab-naming rules) and add one row to `Data_Dictionary`:

form_code	field_name	field_label	field_type	required	validation	description	field_order	widget	help_detail
journal_reviews	review_assigned_date	Review Assigned Date	date	FALSE		Date the editor assigned this manuscript for review	35		

Two things about this row that are easy to get wrong:

- **`field_order` controls display position, not just sort order within the sheet.** The existing `journal_reviews` rows are ordered 10, 20, 30, 40, 50, 60 (`tracking_id` through `notes`). Picking `35` places the new field between `manuscript_id` (30) and `review_date` (40) – logically correct, since a review is assigned before it is completed. Leaving gaps of 10 between existing rows is exactly so a later insertion like this one does not require renumbering anything else.
- **`field_type` in the Sheet is lower case** (`date`), matching every other row. The importer upper-cases it internally before storing it. This matters in Section 2 below, where `add_form_field_by_code()` requires the upper-case form explicitly.

If you maintain a local mirror of the Data Dictionary (this package's own demonstration study keeps one at `vignettes/promotion-tracker/Data_Dictionary.csv`), update it the same way and keep the two in sync:

```
dd <- read.csv('Data_Dictionary.csv', stringsAsFactors = FALSE)

new_row <- dd[1, ]
new_row[1, ]         <- NA
new_row$form_code    <- 'journal_reviews'
new_row$field_name   <- 'review_assigned_date'
new_row$field_label  <- 'Review Assigned Date'
new_row$field_type   <- 'date'
new_row$required     <- FALSE
new_row$validation   <- ''
new_row$description  <- 'Date the editor assigned this manuscript for review'
new_row$field_order  <- 35
new_row$widget       <- ''
```

```r
new_row$help_detail  <- ''

dd2 <- rbind(dd, new_row)
dd2 <- dd2[order(dd2$form_code, dd2$field_order), ]
write.csv(dd2, 'Data_Dictionary.csv', row.names = FALSE)
```

Push the updated data frame to the Sheet with `sheet_write()` (Section 6.1), or edit the row directly in the Sheets UI – either is fine, as long as the two end up identical.

```r
library(googlesheets4)
gs4_auth(email = 'you@example.com')

sheet_write(dd2, ss = 'YOUR_SHEET_ID', sheet = 'Data_Dictionary')
```

---

2. Add the field to the live database

The normal path: `add_form_field_by_code()`

With `DB_ENCRYPTION_KEY` and `ZZEDC_SALT` exported for this session (Section 8 of the technical guide), `add_form_field_by_code()` does the rest of this section in one call: it resolves `form_code` to the form's `design_id`, checks for a duplicate `field_code` on that form, inserts the field, and reads the row back to confirm it was written exactly as requested. **`field_type` must still be the upper-case constant**, from `get_designer_field_types()` – `"DATE"`, not `"date"`. Passing the lower-case Sheet spelling fails with `Invalid field_type: date`; the Google Sheets importer upper-cases it for you on the way in, but this direct call does not.

```r
library(zzedc)

result <- add_form_field_by_code(
  form_code    = 'journal_reviews',       # matched case-insensitively
  field_code   = 'review_assigned_date',
  field_label  = 'Review Assigned Date',
  field_type   = 'DATE',
  field_row    = 35,                      # matches field_order above
  field_column = 1,
  is_required  = FALSE,
  help_text    = 'Date the editor assigned this manuscript for review',
  user_id      = 'your_username'          # attributed in audit_log
)
result$success    # TRUE if the field was added
result$verified    # TRUE if the row read back matches every argument
result$design_id   # resolved from form_code -- no manual lookup needed
```

`user_id` is optional – omitting it still writes the audit row, just without an attributed actor – but passing your own identity here is what makes the resulting `audit_log` entry useful later, the same way `created_by/updated_by/locked_by` already work on this module's other functions.

`field_row` here plays the same role as `field_order` in the Data Dictionary: it is the field's position among its form's other fields, not a literal UI grid row. Giving it the same value you put in the Sheet keeps the two in agreement, which matters if you ever import a fresh copy of this study and compare the two field lists.

Re-running this block by accident is safe: `add_form_field_by_code()` checks for a `field_code` already on that `design_id` before inserting, and returns `result$success == FALSE` with `result$error` naming the existing `field_id` rather than creating a second copy.

```
result$success    # FALSE on a repeat call
result$error       # "Field 'review_assigned_date' already exists on ..."
result$field_id    # the id of the field that already exists
```

An unrecognized `form_code` fails the same way, before anything is written: `result$success == FALSE` with `result$error` reading `"No form found with form_code: ..."`.

If it fails with "Database encryption key not found"

`add_form_field_by_code()` connects through `connect_encrypted_db()` (the same path `add_form_field()` uses), which hard-stops if `DB_ENCRYPTION_KEY` is not set in the current R session – by design, so a stray script can never silently touch an encrypted database it has no business opening. Re-export the key (Section 8) and retry.

If your deployment is one where the database's "encryption" is known not to be doing anything (flagged in `vignette('zzedc-whitepaper')` as an open finding, not a supported mode of operation – track the real fix there, do not treat the workaround below as a substitute for it), a direct `RSQLite` connection reaches the same file without the key check, and you can insert the row by hand with the identical shape `add_form_field_by_code()` would have written. This requires the form's `design_id`, which the escape hatch has no way to resolve for you (that lookup is the part of `add_form_field_by_code()` that needs the key-checked connection):

```
conn <- DBI::dbConnect(RSQLite::SQLite(), 'data/zzedc.db')

design_id <- DBI::dbGetQuery(conn,
  "SELECT design_id FROM designer_forms WHERE UPPER(form_code) = 'JOURNAL_REVIEWS'"
)$design_id[1]

existing <- DBI::dbGetQuery(conn, "
  SELECT field_id FROM designer_fields
  WHERE design_id = ? AND field_code = 'review_assigned_date'
", params = list(design_id))

if (nrow(existing) == 0) {
  DBI::dbExecute(conn, "
    INSERT INTO designer_fields
      (design_id, field_code, field_label, field_type, field_row,
       field_column, field_width, is_required, is_visible, help_text)
    VALUES
      (?, 'review_assigned_date', 'Review Assigned Date', 'DATE', 35, 1,
       100, 0, 1, 'Date the editor assigned this manuscript for review')
  ", params = list(design_id))
}

DBI::dbDisconnect(conn)
```

Treat this as an admin-only escape hatch, not routine practice: it bypasses the package's own validation (`is_backend_available`, field-type checking) and the key-gated connection that `add_form_field_by_code()` enforces. Prefer the normal path above whenever the encryption key is actually available to you.

Only the normal path writes to the audit trail. As of 2026-11-08, `add_form_field()` and `add_form_field_by_code()` log an `INSERT` event to `audit_log` on success (record id, `field_code`/`field_type` in `details`, and `user_id` when you pass it) – this was previously a real gap, documented and now fixed; see `vignette('zzedc-whitepaper')`'s package-boundary section for the full history. The escape hatch above does not get this for free: it writes to `designer_fields` with a plain `RSQLite` connection, which never passes through `add_form_field()`, so a field added that way still leaves no audit row. That is one more reason to prefer the normal path whenever the encryption key is available, beyond the validation it also bypasses.

---

3. Verify

`result$verified` from Section 2 already confirms the database side: it is `TRUE` only when the row read back from `designer_fields` matches every argument passed in (label, type, position, required flag, help text), and `result$mismatches` names exactly which ones did not match on the rare occasion it is `FALSE`. That rules the database out as a cause if the field still does not show up live – at that point the cause is the running browser tab, not the data, so the one thing left to check by hand is the UI rendering itself.

No application restart and no package reinstall are needed. Unlike an R code change (which requires `R CMD INSTALL` and a fresh R session, per Section 12's note on `library()` being a no-op when the package is already attached), a `designer_fields` row is read fresh from the database on every page load – the dynamic form renderer queries it live.

Reload the EDC tab in the browser, switch the **Form** selector to **JOURNAL_REVIEWS Form**, and confirm:

- **Review Assigned Date** appears as a date field, positioned between Manuscript ID and Review Date.
- It has no red required-asterisk (it was added with `is_required = FALSE`).
- Clicking its hint icon shows the help text: "Date the editor assigned this manuscript for review."

If the field does not appear, the most likely cause is a stale reactive cache in an already-open browser tab rather than anything wrong with the database row – a hard reload (not just re-clicking the form tab) forces the metadata query to re-run.

---

4. What this walkthrough deliberately did not do

- **No validation rule was added** (e.g. requiring `review_date >= review_assigned_date`). Nothing in the original request called for one; see `vignette('content-author-guide')` for the DSL syntax if you want to add one later, following the pattern of `MENTOR_END_AFTER_START` / `COMM_END_AFTER_START` in the technical guide's validation-rules table (Section 6.1).
- **The field was not marked required.** Existing `journal_reviews` records have no value for it and would fail validation retroactively if it were.
- **No backup was taken before this change.** Adding a column is additive and reversible (`DROP COLUMN` or simply ignoring the new field in the UI), unlike the destructive `overwrite = TRUE` rebuild this guide opened by warning against – but Section 9 of the technical guide's backup routine costs little and is worth running on a schedule regardless.

Rendered on 2026-11-08 at 16:51 PST. Source: ~/prj/sfw/05-zzedc/zzedc/vignettes/promotion-tracker/promotion-tracker-add-field-walkthrough.Rmd